

Pate02

2- what's the difference between compiled and interpreted languages and in this way what about Csharp?

- **Compiled Languages (e.g., C++, C):** Translated entirely into machine code **before** running. They are fast, but platform-dependent.
- **Interpreted Languages (e.g., Python, JS):** Translated line-by-line **during** execution. They are slower, but easy to run anywhere.

Where does C# fit in?

C# uses a **hybrid approach**:

1. **At Build Time:** The compiler turns your code into an intermediate format called **IL (Intermediate Language)**.
2. **At Runtime:** The **CLR** uses a **JIT (Just-In-Time) Compiler** to translate that IL into native machine code right as the application runs.

This gives C# the **speed** of compiled languages with the **flexibility** of interpreted ones.

3- Compare between implicit, explicit, Convert and parse casting

Implicit Casting Happens automatically by the compiler without any extra syntax. It occurs when converting a smaller data type into a larger data type where there is no risk of data loss.

- **Key detail:** Safe and seamless (e.g., converting an `int` to a `double`).

Explicit Casting Requires manual intervention using the cast operator like `(int)`. It is necessary when converting a larger data type into a smaller one, where precision or data loss can occur.

- **Key detail:** Truncates data if the value exceeds the target type's range (e.g., casting a `double` to an `int` cuts off the decimal part).

Parse Method Designed specifically to convert **strings** into target primitive data types (e.g., `int.Parse("123")`).

- **Key detail:** Strict and fragile. It throws an `ArgumentNullException` if the input is `null` and a `FormatException` if the string contains invalid characters. (Use `TryParse` as a safe alternative to avoid crashes).

Convert Class A helper class containing static methods (e.g., `Convert.ToInt32()`) to convert between completely different data types.

- **Key detail:** Null-safe. If you pass a `null` value to `Convert`, it handles it gracefully without throwing an exception and returns the target type's default value (such as `0` for numbers).

Part03 Bonus

*Self:

1. What is the difference between `_firstName` and `firstName` in C#, and when should we use each one?

`_firstName` (Private Field)

- **When to use:** Used for **private fields** declared at the class level.
- **Why:** The underscore prefix `_` clearly indicates that the variable is private to the class, avoiding the need to write `this.firstName` to resolve ambiguity.
- **Format:** `_camelCase` (e.g., `_firstName`, `_dbContext`).

`firstName` (Local Variable / Parameter)

- **When to use:** Used for **method parameters** or **local variables** declared inside a method/constructor.
- **Why:** Standard C# naming convention for scoped variables that are not class-level fields.
- **Format:** `camelCase` (e.g., `firstName`, `totalAmount`).

2. What causes memory leaks in C# when holding references to `this`?

Memory Leak Avoidance (`this` retention): Occurs when an object is kept alive in memory because another long-lived component holds a strong reference to it (`this`).

- **Prevention:** Always unsubscribe from events (`-=`), clear long-lived static references, and properly dispose of unmanaged resources using `using` statements or `IDisposable`.

3. Stack vs Heap vs Data Structures (Nature of Memory)?

1. Stack Memory

- **Nature:** Fast, small, and managed automatically using **LIFO** (Last-In, First-Out).
- **What it stores:** **Value Types** (`int`, `double`, `bool`, `struct`) and local reference pointers.
- **Cleanup:** Cleared immediately when the method scope ends.

2. Heap Memory

- **Nature:** Large and flexible, but slightly slower access.
- **What it stores: Reference Types** (`class`, `string`, `array`, `object`).
- **Cleanup:** Managed by the **Garbage Collector (GC)** when objects are no longer referenced.

3. Data Structures in Memory

- **Arrays & Lists:** The container object itself lives on the **Heap**.
- **Structs:** Stored directly on the **Stack** (unless boxed or part of a class), making them lightweight.

Quick Comparison:

- **Stack:** Fast | Small | Value Types | Auto-cleaned on scope exit
 - **Heap:** Slower | Large | Reference Types | Cleaned by Garbage Collector
-

what meant by Csharp is managed code?

C# is called **managed code** because its execution is entirely managed by a runtime engine called the **Common Language Runtime (CLR)** rather than executing directly on the operating system's hardware.

- **Intermediate Compilation:** C# code is not compiled directly into native machine instructions. Instead, it compiles into **IL (Intermediate Language)** bytecode.
- **JIT Compilation:** At runtime, the CLR's **Just-In-Time (JIT)** compiler translates this IL into native CPU instructions right before execution.
- **Automatic Memory Management:** The CLR handles memory allocation and runs the **Garbage Collector (GC)** to automatically free unused memory.
- **Type Safety & Security:** The CLR enforces strict type checks, array bounds checking, and pointer protection to prevent memory corruption and unauthorized memory access.
- **Exception Handling:** Program errors and crashes are trapped and managed safely through structured runtime exception handling.

Managed vs. Unmanaged Code:

- **Managed Code (C#, VB.NET, F#):** Runs inside the CLR environment, benefits from automatic garbage collection, memory safety, and cross-platform flexibility.
- **Unmanaged Code (C, C++):** Compiles directly to native binary machine code for a specific CPU architecture. The developer must manually allocate and free memory (using `malloc/free` or `new/delete`), making it faster but prone to memory leaks and pointer errors.

what meant by struct is considered like class before?

Saying "**a struct is like a class**" means they look identical in syntax:

- Both can have methods, constructors, properties, and interfaces.

The Critical Difference:

- **class (Reference Type):** Stored on the **Heap**. Assigning it creates a **reference pointer** to the same memory location.
- **struct (Value Type):** Stored on the **Stack**. Assigning it creates an **independent copy** of the data.
- **Inheritance:** Classes support inheritance; structs do not.